Comprehensive DBMS Project Solutions

A Detailed Guide for Computer Science & Engineering, 3rd Year (2025)

Generated on: 2025-12-11

Disclaimer: This document serves as an extensive educational guide based on the project requirements outlined in the `DBMS_Project_for_CSE_3rd_Year (1).pdf` file. The code snippets and architectural patterns provided are illustrative examples intended to guide students in building their own solutions. They are not a complete, copy-paste solution but a foundation for learning and implementation. Students are encouraged to adapt, expand, and innovate upon these designs.

IntroductionCore PrinciplesProject Solutions



Introduction

This document provides a comprehensive set of solutions and technical guidelines for the Database Management Systems (DBMS) projects designated for third-year Computer Science and Engineering students in 2025. The original project brief outlines a series of twelve distinct web application projects, each designed to test and develop students' skills in database design, full-stack development, and system architecture.

The primary reference for this guide is the course document `DBMS_Project_for_CSE_3rd_Year (1).pdf`, which specifies project rules, technology options, and functional requirements for each system.

Project Guidelines Summary

Before diving into the solutions, let's recap the key rules and constraints from the project document:

- Group Size: A maximum of 5 students per group.
- Registration Deadline: All groups must register their members and chosen technology stack by October 23, 2025.
- Technology Constraints: No more than two groups are permitted to select the same technology option. Allocations are on a first-come, first-served basis.
- Version Control: Each group is required to create and maintain a GitLab repository for their project.

Available Technology Stacks

The document provides a flexible set of modern technology stacks. Students must choose one combination from the following options. This guide will primarily focus on providing detailed examples using the React + Express (Node.js) stack due to its popularity and versatility, but will also offer high-level architectural guidance for the other frameworks.

Option	Frontend Framework	Backend Framework	Type
1	React	Express (Node.js)	Separated Frontend/Backend
2	Vue.js	Express (Node.js)	Separated Frontend/Backend
3	Angular	Express (Node.js)	Separated Frontend/Backend
4	-	Laravel (PHP)	Full-stack Framework (e.g., with Blade or React/Vue)
5	-	ASP.NET (C#)	Full-stack Framework (e.g., with Razor Pages or React/Angular)

Option	Frontend Framework	Backend Framework	Type
6	-	Django (Python)	Full-stack Framework (e.g., with DTL or React/Vue)



Core Principles for Robust Application Development

Before we address each project individually, it's crucial to understand the foundational principles that apply to all of them. A strong grasp of database design, security, and architecture will ensure your project is not only functional but also scalable, maintainable, and secure. These principles are the hallmark of professional software engineering.

Database Design Best Practices

The database is the heart of your application. A well-designed database simplifies development, enhances performance, and ensures data integrity. Conversely, a poorly designed database can lead to complex queries, slow performance, and data anomalies. Database design basics are fundamental to success.

1. Entity-Relationship (ER) Modeling

Before writing a single line of SQL, you should model your data using an Entity-Relationship Diagram (ERD). An ERD is a visual blueprint of your database schema. It helps you identify the main entities, their attributes, and the relationships between them. The ER data model is based on two core concepts: entities (tables) and relationships (associations between tables).

- Entities: Represent real-world objects or concepts (e.g., `Student`, `Course`, `Product`). In an ERD, these are typically shown as rectangles.
- Attributes: Represent the properties of an entity (e.g., a `Student` has a `name`, `email`, and `student_id`). These are often listed within the entity's rectangle.
- Relationships: Define how entities are connected. The main types are:

    - One-to-One (1:1): Each record in Table A is linked to one and only one record in Table B. (e.g., `User` and `UserProfile`).
    - One-to-Many (1:N): One record in Table A can be linked to many records in Table B, but a record in Table B is linked to only one record in Table A. (e.g., one `Author` can have many `Books`).
    - Many-to-Many (M:N): A record in Table A can be linked to many records in Table B, and vice-versa. (e.g., `Students` and `Courses`). This relationship is implemented using a "junction" or "associative" table.
- Keys:
    - Primary Key (PK): An attribute (or set of attributes) that uniquely identifies each record in a table. It cannot be null. In ERDs, it's often underlined.
    - Foreign Key (FK): A key used to link two tables together. It's a field in one table that refers to the Primary Key in another table.

Creating an ERD forces you to think through your data structure logically, preventing design flaws before they become costly to fix. Tools like Creately, EdrawMax, or even simple diagramming software can be used to create ERDs. For a university system, an ERD helps visualize connections between students, courses, and enrollments, ensuring a scalable design .

2. Database Normalization

Normalization is the process of organizing tables and columns in a relational database to minimize data redundancy and improve data integrity. As explained by DigitalOcean, it involves a series of "normal forms" (NF), with the first three being the most important for most applications.

- First Normal Form (1NF): Ensures that a table's columns hold atomic (indivisible) values and that each row is unique. This means no repeating groups or multi-valued columns. For example, instead of storing multiple phone numbers in a single `phone_numbers` column, you should create a separate `PhoneNumbers` table linked to the user.
- Second Normal Form (2NF): Requires the table to be in 1NF and all non-key attributes to be fully functionally dependent on the entire primary key. This rule

applies to tables with composite primary keys. It prevents partial dependencies, where an attribute depends on only part of the composite key.

- Third Normal Form (3NF): Requires the table to be in 2NF and all attributes to be dependent only on the primary key, not on other non-key attributes. This eliminates transitive dependencies. For example, if you have a `Books` table with `author_name` and `author_email`, the `author_email` depends on `author_name`, which is not the primary key. The author information should be moved to a separate `Authors` table.

Proper normalization leads to a more flexible, less redundant, and more maintainable database schema, which is a core requirement for all the projects listed.

3. Indexing Strategies

Database indexes are data structures that improve the speed of data retrieval operations on a database table at the cost of additional writes and storage space. As Acceldata highlights, indexing is the cornerstone of fast data retrieval.

- When to Index: Create indexes on columns that are frequently used in `WHERE` clauses, `JOIN` conditions, and `ORDER BY` clauses. Primary keys are automatically indexed. Foreign keys are excellent candidates for indexing.
- Types of Indexes:
  - Clustered Index: Determines the physical order of data in a table. A table can have only one clustered index.
  - Non-Clustered Index: Has a separate structure from the data rows. It contains the index key values and a pointer to the corresponding data row. A table can have multiple non-clustered indexes.
  - Composite Index: An index on two or more columns. Useful for queries that filter on multiple conditions.
- Trade-offs: While indexes speed up `SELECT` queries, they can slow down data modification operations (`INSERT`, `UPDATE`, `DELETE`) because the indexes also need to be updated. Avoid over-indexing, especially on write-heavy tables.

Web Application Security Best Practices

Security is not an afterthought; it must be integrated into every stage of the development lifecycle (a practice known as DevSecOps). The OWASP Top 10 provides a standard awareness document for the most critical web application security risks. All projects must address these common vulnerabilities.

1. Authentication and Authorization

These two terms are related but distinct.

- Authentication: The process of verifying who a user is. This is typically done via a login form (username/password), but can also involve multi-factor authentication (MFA) or social logins.
- Authorization: The process of determining what an authenticated user is allowed to do. This is where Role-Based Access Control (RBAC) comes in.

For your projects, you will need to implement robust authentication. For example, passwords should never be stored in plaintext. They must be hashed using a strong, one-way algorithm like Argon2 or bcrypt. The OWASP Authentication Cheat Sheet is an excellent resource for best practices.

2. Role-Based Access Control (RBAC)

Nearly every project in the list specifies role-based access (e.g., admin, doctor, student, guest). RBAC is a method of restricting system access to authorized users based on their roles. Implementing RBAC involves:

1. Defining Roles: Identify the different user roles in your system (e.g., `Admin`, `Librarian`, `Member` for a Library System).
2. Defining Permissions: Identify the actions that can be performed (e.g., `create_book`, `delete_member`, `issue_book`).
3. Assigning Permissions to Roles: Map which roles can perform which actions. An `Admin` might have all permissions, while a `Librarian` can `create_book` and `issue_book`, but not `delete_member`.
4. Assigning Roles to Users: Each user in your system is assigned one or more roles.
5. Enforcing Permissions: In your application's code (e.g., in API middleware or controller logic), check if the current user's role has the necessary permission before allowing an action to proceed.

Client-side RBAC (like hiding a button in the UI) is for user experience, but true security is enforced on the server-side. Never trust the client. Microsoft's guide on RBAC emphasizes that client-side checks must be paired with server-side enforcement.

3. Preventing Common Vulnerabilities

- SQL Injection (SQLi): This attack occurs when malicious SQL code is inserted into an application's input, allowing an attacker to interfere with database queries.
    - Prevention: Never construct SQL queries by concatenating strings with user input. Always use **parameterized queries** (also known as prepared statements). Modern ORMs (Object-Relational Mappers) like Sequelize, Django ORM, Eloquent, and Entity Framework Core do this for you automatically, which is a major reason to use them. This guide on fixing SQLi explains the importance of parameterized queries.
- Cross-Site Scripting (XSS): XSS attacks inject malicious scripts into web pages viewed by other users.
    - Prevention: The primary defense is **output encoding**. When displaying user-provided content in your HTML, ensure that special characters (like `<` and `>`) are properly escaped so the browser treats them as literal text, not as HTML tags. Most modern frontend frameworks (like React) and backend templating engines (like Blade or Jinja2) do this automatically. Be careful when you need to intentionally render HTML (e.g., using `dangerouslySetInnerHTML` in React) and use a sanitization library like DOMPurify. MDN provides a great overview of XSS prevention.
- Cross-Site Request Forgery (CSRF): CSRF attacks trick a logged-in user into submitting a malicious request to a web application they are authenticated with.
    - Prevention: Use anti-CSRF tokens. The server generates a unique, unpredictable token for each user session and embeds it in forms. When a form is submitted, the server checks if the submitted token matches the one stored in the session. Most full-stack frameworks like Django, Laravel, and ASP.NET have built-in CSRF protection. For SPAs with a separate backend, other techniques like using custom request headers are common.
- Secure File Uploads: Allowing users to upload files is risky. An attacker could upload a malicious script, an oversized file to cause a denial-of-service, or a file that exploits

a vulnerability in the server's file processing libraries.

- Prevention: Follow a strict checklist as recommended by OWASP's File Upload Cheat Sheet:
  1. Validate the file type on the server (don't trust the `Content-Type` header or file extension).
  2. Generate a new, random filename for the stored file to prevent path traversal attacks.
  3. Set a reasonable file size limit.
  4. Store uploaded files outside of the web root, or on a separate file server (like AWS S3).
  5. Scan uploaded files for viruses.

4. Session Management

HTTP is a stateless protocol. To maintain a user's logged-in state across multiple requests, we use sessions. Secure session management is critical to prevent session hijacking.

- Session IDs: After a user logs in, the server creates a session and gives the client a unique session ID, usually stored in a cookie. This ID should be long, random, and unpredictable.
- Secure Cookies: Session cookies should be configured with security attributes:
  - `HttpOnly`: Prevents the cookie from being accessed by client-side JavaScript, mitigating XSS attacks.
  - `Secure`: Ensures the cookie is only sent over HTTPS.
  - `SameSite=Strict` or `SameSite=Lax`: Helps protect against CSRF attacks.
- Server-Side vs. Client-Side Storage:
  - Server-Side Sessions (Recommended for web apps): The session ID is just a pointer. All sensitive data is stored on the server (e.g., in Redis or a database). This is more secure. Flask-Session with Redis is a common pattern.

    - Client-Side Tokens (JWTs for SPAs/APIs): JSON Web Tokens (JWTs) are a popular way to handle authentication in Single-Page Applications (SPAs). The server issues a signed token containing user information (the payload). The client stores this token (e.g., in memory or `localStorage`) and sends it with every API request in an `Authorization` header. The server verifies the token's signature to authenticate the request. While convenient, storing JWTs in `localStorage` makes them vulnerable to XSS. Storing them in `HttpOnly` cookies is a more secure alternative.

API Design Principles

Most of these projects will require a backend API to serve data to a frontend application. Designing a clean, predictable, and secure API is essential.

- REST (Representational State Transfer): REST is an architectural style for designing networked applications. A RESTful API uses standard HTTP methods (`GET`, `POST`, `PUT`, `DELETE`) to perform CRUD (Create, Read, Update, Delete) operations on resources.
    - `GET /api/books`: Retrieve a list of all books.
    - `GET /api/books/123`: Retrieve a specific book with ID 123.
    - `POST /api/books`: Create a new book.
    - `PUT /api/books/123`: Update the book with ID 123.
    - `DELETE /api/books/123`: Delete the book with ID 123.
- Statelessness: Each request from a client to the server must contain all the information needed to understand and complete the request. The server should not store any client context between requests. Authentication is typically handled by sending a token with each request.
- JSON as a Data Format: JSON (JavaScript Object Notation) is the de facto standard for data exchange in modern web APIs due to its lightweight nature and ease of parsing in JavaScript.
- API Documentation: Good APIs are well-documented. Tools like Swagger/OpenAPI can automatically generate interactive API documentation from your code, which is invaluable for frontend developers and for testing. Setting up Swagger in Spring Boot is a common practice.

Project Solutions

Table of Contents

- 1. Hotel Reservation System
- 2. Inventory Management System
- 3. Hospital Management System
- 4. Library Management System
- 5. Restaurant Management System
- 6. HR Management System
- 7. Online Rental System
- 8. Tourist Management System
- 9. School Management System
- 10. Online Food Ordering System
- 11. Blood Bank Management System
- 12. Pharmacy Management System

1. Hotel Reservation System

1.1 Functional Requirements

The system must support the management of hotel operations, focusing on guests, rooms, and bookings. Key features include:

- Guest Management: Add new guest details (name, contact info, address), and update or delete existing guest information.

- Room Management: Add new rooms (number, type, availability, price), and update room details.
- Booking Management: Book a room for a guest for specific check-in/check-out dates, modify existing reservations, and cancel reservations.
- Authentication & Authorization: Implement role-based access for Admin, Receptionist, and Guest roles, with secure login and logout functionality.
- Reporting: Generate daily, weekly, or monthly reports on bookings and occupancy.

1.2 Database Design

A relational database is ideal for this system due to the structured nature of the data and the clear relationships between entities. The design focuses on normalizing data to reduce redundancy and ensure integrity, for example, by separating users, rooms, and bookings into distinct tables. Similar designs for booking apps provide a good reference.

Entity-Relationship Diagram (ERD) Explanation

- Users: Stores information about all individuals interacting with the system, including guests, receptionists, and admins. A `role` column will differentiate them.
- Rooms: Contains details about each room in the hotel, such as room number, type, price, and capacity.
- RoomTypes: A separate table to define room categories (e.g., 'Single', 'Double', 'Suite') and their base properties. This normalizes the `Rooms` table.
- Bookings: The central table linking users (guests) to rooms. It records check-in/out dates, booking status, and total cost.
- Payments: Tracks payments associated with each booking. This is a one-to-one relationship with a booking.

The relationship between `Users` and `Bookings` is one-to-many (one user can have many bookings). The relationship between `Rooms` and `Bookings` is also one-to-many (one room can be part of many bookings over time, but not concurrently).

Database Schema (SQL)

```sql
-- Users Table: Stores guests, receptionists, and admins
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    phone_number VARCHAR(20),
    address TEXT,
    role ENUM('guest', 'receptionist', 'admin') NOT NULL DEFAULT 'guest',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- RoomTypes Table: Defines categories of rooms
CREATE TABLE RoomTypes (
    room_type_id INT AUTO_INCREMENT PRIMARY KEY,
    type_name VARCHAR(50) NOT NULL UNIQUE, -- e.g., 'Standard Single', 'Delu
    description TEXT,
    base_price DECIMAL(10, 2) NOT NULL,
    capacity INT NOT NULL
);

-- Rooms Table: Stores individual room details
CREATE TABLE Rooms (
    room_id INT AUTO_INCREMENT PRIMARY KEY,
    room_number VARCHAR(10) NOT NULL UNIQUE,
    room_type_id INT NOT NULL,
    status ENUM('available', 'occupied', 'maintenance') NOT NULL DEFAULT 'av
    FOREIGN KEY (room_type_id) REFERENCES RoomTypes(room_type_id)
);

-- Bookings Table: Manages all reservations
CREATE TABLE Bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    guest_id INT NOT NULL,
    room_id INT NOT NULL,
    check_in_date DATE NOT NULL,
    check_out_date DATE NOT NULL,
    num_guests INT NOT NULL,
    total_price DECIMAL(10, 2) NOT NULL,
    status ENUM('confirmed', 'checked-in', 'checked-out', 'cancelled') NOT N
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTA
```

```
    FOREIGN KEY (guest_id) REFERENCES Users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (room_id) REFERENCES Rooms(room_id),
    CONSTRAINT chk_dates CHECK (check_out_date > check_in_date)
);

-- Payments Table: Tracks payment information for each booking
CREATE TABLE Payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT NOT NULL UNIQUE,
    amount DECIMAL(10, 2) NOT NULL,
    payment_method VARCHAR(50),
    payment_status ENUM('pending', 'completed', 'failed') NOT NULL DEFAULT '
    transaction_id VARCHAR(100),
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (booking_id) REFERENCES Bookings(booking_id) ON DELETE CASCA
);

-- Create indexes for performance
CREATE INDEX idx_bookings_dates ON Bookings(check_in_date, check_out_date);
CREATE INDEX idx_bookings_guest ON Bookings(guest_id);
CREATE INDEX idx_rooms_status ON Rooms(status);
```

1.3 Implementation Guide (React + Node.js/Express)

This section provides a high-level guide to building the Hotel Reservation System using React for the frontend and Node.js with the Express framework for the backend API. We'll use a SQL database (like MySQL or PostgreSQL) with the Sequelize ORM for data access.

Backend (Node.js, Express, Sequelize)

1. Project Setup

Initialize a Node.js project and install necessary packages.

```
mkdir hotel-api
cd hotel-api
npm init -y
```

```
npm install express sequelize mysql2 jsonwebtoken bcryptjs cors dotenv
```

2. Folder Structure

A modular structure helps in maintaining the codebase.

```
/hotel-api
|-- config/
|   |-- database.js       # Sequelize connection
|-- controllers/
|   |-- authController.js   # Handles login, registration
|   |-- bookingController.js # Handles booking logic
|   |-- roomController.js     # Handles room CRUD
|-- middleware/
|   |-- authMiddleware.js   # Protects routes, checks roles
|-- models/
|   |-- user.js
|   |-- room.js
|   |-- booking.js
|   |-- ... (other models)
|-- routes/
|   |-- authRoutes.js
|   |-- bookingRoutes.js
|   |-- ... (other routes)
|-- .env                  # Environment variables (DB_USER, JWT_SECRET)
|-- server.js             # Main server entry point
```

3. Authentication and Authorization

Authentication can be implemented using JSON Web Tokens (JWT). When a user logs in, the server generates a signed JWT containing the user's ID and role. This token is sent back to the client, which includes it in the `Authorization` header for subsequent requests. This tutorial from Bezkoder provides an excellent step-by-step guide.

The `authMiddleware.js` will verify the token and can also check for roles.

```
// middleware/authMiddleware.js
```

```
const jwt = require('jsonwebtoken');

const protect = (req, res, next) => {
    let token;
    if (req.headers.authorization && req.headers.authorization.startsWith('B
        try {
            token = req.headers.authorization.split(' ')[1];
            const decoded = jwt.verify(token, process.env.JWT_SECRET);
            req.user = decoded; // Adds user info (id, role) to the request
            next();
        } catch (error) {
            res.status(401).json({ message: 'Not authorized, token failed' }
        }
    }
    if (!token) {
        res.status(401).json({ message: 'Not authorized, no token' });
    }
};

const authorize = (...roles) => {
    return (req, res, next) => {
        if (!roles.includes(req.user.role)) {
            return res.status(403).json({ message: `User role ${req.user.rol
        }
        next();
    };
};

module.exports = { protect, authorize };
```

4. API Endpoints (RESTful)

Define routes in the `routes` folder and link them to controller functions. Protect routes using the middleware.

```
// routes/bookingRoutes.js
const express = require('express');
const router = express.Router();
const { createBooking, getBookings, cancelBooking } = require('../controller
const { protect, authorize } = require('../middleware/authMiddleware');
```

```
// All users can view their own bookings, but only receptionists/admins can
router.route('/')
    .post(protect, authorize('admin', 'receptionist'), createBooking)
    .get(protect, getBookings);

router.route('/:id/cancel')
    .put(protect, cancelBooking); // A guest can cancel their own booking

module.exports = router;
```

5. Controller Logic

Controllers contain the business logic. They interact with models (Sequelize) to perform database operations.

```
// controllers/bookingController.js
const { Booking, Room, User } = require('../models'); // Assuming models are

// @desc   Create a new booking
// @route  POST /api/bookings
// @access Private (Admin, Receptionist)
exports.createBooking = async (req, res) => {
    const { guest_id, room_id, check_in_date, check_out_date, num_guests } =

    try {
        // 1. Check if the room is available for the given dates (complex lo
        // This would involve querying the Bookings table for overlapping da

        // 2. Calculate total price (logic omitted)
        const totalPrice = 200.00; // Placeholder

        const booking = await Booking.create({
            guest_id,
            room_id,
            check_in_date,
            check_out_date,
            num_guests,
            total_price: totalPrice,
            status: 'confirmed'
```

```
        });

        // 3. Update room status if necessary
        await Room.update({ status: 'occupied' }, { where: { room_id } });

        res.status(201).json(booking);
    } catch (error) {
        res.status(500).json({ message: 'Server Error', error: error.message
    }
};
```

Frontend (React)

1. Project Setup

Use Create React App or Vite to bootstrap the project.

```
npm create vite@latest hotel-client -- --template react
cd hotel-client
npm install
npm install axios react-router-dom
```

2. Component Structure

Organize the UI into logical components.

```
/src
|-- components/
|   |-- BookingForm.js
|   |-- RoomList.js
|   |-- Navbar.js
|   |-- ReportGenerator.js
|-- pages/
|   |-- HomePage.js
|   |-- LoginPage.js
|   |-- RegisterPage.js
|   |-- DashboardPage.js
```

```
|   |-- BookingsPage.js
|-- services/
|   |-- api.js            # Axios instance and API calls
|-- App.js
|-- index.js
```

3. API Service

Centralize API calls using Axios. This makes it easy to manage base URLs and attach the JWT token to headers.

```
// services/api.js
import axios from 'axios';

const API = axios.create({
    baseURL: 'http://localhost:5000/api' // Your backend URL
});

API.interceptors.request.use((req) => {
    const user = JSON.parse(localStorage.getItem('user'));
    if (user && user.token) {
        req.headers.Authorization = `Bearer ${user.token}`;
    }
    return req;
});

export const login = (formData) => API.post('/auth/login', formData);
export const createBooking = (bookingData) => API.post('/bookings', bookingD
export const fetchRooms = () => API.get('/rooms');
```

4. Example Component: Booking Form

A form component to create a new booking, which calls the API service on submission.

```
// components/BookingForm.js
import React, { useState } from 'react';
import { createBooking } from '../services/api';
```

```
const BookingForm = () => {
    const [formData, setFormData] = useState({
        room_id: '',
        check_in_date: '',
        check_out_date: ''
    });

    const handleChange = (e) => {
        setFormData({ ...formData, [e.target.name]: e.target.value });
    };

    const handleSubmit = async (e) => {
        e.preventDefault();
        try {
            const response = await createBooking(formData);
            alert('Booking created successfully!');
            // Handle success (e.g., redirect or clear form)
        } catch (error) {
            alert('Failed to create booking.');
            console.error(error);
        }
    };

    return (
        <form onSubmit={handleSubmit}>
            {/* Form inputs for room_id, dates, etc. */}
            <button type="submit">Book Room</button>
        </form>
    );
};

export default BookingForm;
```

1.4 Guidance for Other Frameworks

- Django: Use Django REST Framework (DRF) to build the API. Django's built-in ORM and admin panel are powerful. Authentication can be handled with DRF's `TokenAuthentication` or `django-rest-framework-simplejwt`. Models would be defined in `models.py`, views (controllers) in `views.py`, and URL routing in `urls.py`.

- Laravel: A full-stack PHP framework. You can build a REST API using its routing system and Eloquent ORM. For authentication, Laravel Sanctum is perfect for SPAs, providing a lightweight token-based system. This guide on building a CRUD API with Laravel and Sanctum is a great starting point. You can serve a React frontend separately or integrate it into your Laravel project using Inertia.js.
- ASP.NET Core: A robust, high-performance framework from Microsoft. Use ASP.NET Core Web API to create RESTful services. Entity Framework Core is the standard ORM. Authentication can be implemented using ASP.NET Core Identity with JWT Bearer tokens. This tutorial shows how to build a secure CRUD app with ASP.NET Core and React.



2. Inventory Management System

2.1 Functional Requirements

This system is designed to track products, suppliers, and stock levels within a warehouse or store.

- Product Management: Add, update, view, and delete products with details like name, category, price, quantity, and supplier.
- Supplier Management: Add, update, and delete supplier information (name, contact details, address).
- Stock Control: Record incoming stock (purchase orders) and track outgoing stock (sales). Handle stock adjustments for damaged or returned goods.
- Organization: Organize products by categories. Add, update, or delete categories.
- Reporting & Tools: Generate stock reports (e.g., low-stock items) and generate/scan barcodes or QR codes for products.
- Access Control: Role-based access for Admin, Inventory Manager, and Viewer roles.

2.2 Database Design

The database for an inventory system must be robust in tracking the movement of stock. The design separates products, suppliers, and transactions to maintain a clear and

auditable history of inventory changes. GeeksforGeeks provides a solid foundation for such a design.

Entity-Relationship Diagram (ERD) Explanation

- Users: Stores user accounts with roles (`admin`, `manager`, `viewer`).
- Products: The core entity, containing details about each item in the inventory.
- Categories: A table to group products, linked to `Products` with a foreign key.
- Suppliers: Stores information about vendors from whom products are purchased.
- PurchaseOrders: Represents an order placed with a supplier to procure stock.
- PurchaseOrderItems: A junction table detailing which products and in what quantities are included in a purchase order.
- SalesOrders: Represents a customer sale.
- SalesOrderItems: A junction table for products included in a sales order.
- StockAdjustments: A log table to record manual changes to stock levels (e.g., for damaged goods, returns, or physical count corrections).

Database Schema (SQL)

```
-- Users Table
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('admin', 'manager', 'viewer') NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Suppliers Table
CREATE TABLE Suppliers (
    supplier_id INT AUTO_INCREMENT PRIMARY KEY,
    supplier_name VARCHAR(100) NOT NULL,
    contact_person VARCHAR(100),
    email VARCHAR(100),
```

```
    phone_number VARCHAR(20),
    address TEXT
);

-- Categories Table
CREATE TABLE Categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT
);

-- Products Table
CREATE TABLE Products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    sku VARCHAR(50) UNIQUE, -- Stock Keeping Unit
    category_id INT,
    supplier_id INT,
    price DECIMAL(10, 2) NOT NULL,
    quantity_on_hand INT NOT NULL DEFAULT 0,
    reorder_level INT DEFAULT 10,
    barcode VARCHAR(100) UNIQUE,
    FOREIGN KEY (category_id) REFERENCES Categories(category_id),
    FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id)
);

-- StockTransactions Table: A single ledger for all stock movements
CREATE TABLE StockTransactions (
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    product_id INT NOT NULL,
    transaction_type ENUM('purchase', 'sale', 'adjustment_in', 'adjustment_o
    quantity INT NOT NULL,
    transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    notes TEXT,
    related_order_id INT, -- Optional: links to a purchase or sale order
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

-- PurchaseOrders Table
CREATE TABLE PurchaseOrders (
    po_id INT AUTO_INCREMENT PRIMARY KEY,
    supplier_id INT NOT NULL,
    order_date DATE NOT NULL,
```

```
    expected_delivery_date DATE,
    status ENUM('pending', 'ordered', 'received', 'cancelled') NOT NULL,
    FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id)
);

-- PurchaseOrderItems Table
CREATE TABLE PurchaseOrderItems (
    po_item_id INT AUTO_INCREMENT PRIMARY KEY,
    po_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10, 2),
    FOREIGN KEY (po_id) REFERENCES PurchaseOrders(po_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

-- Indexes for performance
CREATE INDEX idx_products_name ON Products(name);
CREATE INDEX idx_products_category ON Products(category_id);
CREATE INDEX idx_stocktransactions_product ON StockTransactions(product_id);
```

> Design Note: Instead of separate tables for incoming/outgoing stock, a single `StockTransactions` ledger provides a more robust and auditable system. Every change to a product's quantity is recorded as a transaction. The `quantity_on_hand` in the `Products` table can be calculated from this ledger or updated via triggers for faster reads.

2.3 Implementation Guide (Django + React)

For this project, we'll outline an implementation using Django for the backend and React for the frontend. Django's powerful ORM and built-in admin panel are exceptionally well-suited for data-intensive applications like an inventory system.

Backend (Django & Django REST Framework)

1. Project Setup

Set up a Django project and create an `inventory` app.

```
pip install django djangorestframework djangorestframework-simplejwt
django-admin startproject inventory_project
cd inventory_project
python manage.py startapp inventory
```

Add `rest_framework`, `rest_framework_simplejwt`, and `inventory` to `INSTALLED_APPS` in `settings.py`.

2. Models

Define your database schema in `inventory/models.py`. The Django ORM will translate these Python classes into database tables. This guide on Django and React provides a good overview of model creation.

```
# inventory/models.py
from django.db import models
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    # Add custom fields if needed, roles can be handled by Django's Group mo
    pass

class Category(models.Model):
    name = models.CharField(max_length=100, unique=True)
    description = models.TextField(blank=True, null=True)

    def __str__(self):
        return self.name

class Product(models.Model):
    name = models.CharField(max_length=100)
    sku = models.CharField(max_length=50, unique=True)
    description = models.TextField(blank=True, null=True)
    category = models.ForeignKey(Category, on_delete=models.SET_NULL, null=T
    price = models.DecimalField(max_digits=10, decimal_places=2)
    quantity_on_hand = models.IntegerField(default=0)
```

```
    def __str__(self):
        return self.name
# ... other models like Supplier, StockTransaction, etc.
```

After defining models, create and run migrations: `python manage.py makemigrations` and `python manage.py migrate`.

3. Serializers

Serializers in DRF convert complex data types, like Django model instances, into native Python datatypes that can then be easily rendered into JSON.

```
# inventory/serializers.py
from rest_framework import serializers
from .models import Product, Category

class CategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = Category
        fields = '__all__'

class ProductSerializer(serializers.ModelSerializer):
    category_name = serializers.CharField(source='category.name', read_only=

    class Meta:
        model = Product
        fields = ['id', 'name', 'sku', 'description', 'price', 'quantity_on_
```

4. Views and URLs

Use DRF's `ModelViewSet` to quickly create a full set of CRUD endpoints for your models.

```
# inventory/views.py
from rest_framework import viewsets, permissions
from .models import Product, Category
from .serializers import ProductSerializer, CategorySerializer
```

```
class IsAdminOrManager(permissions.BasePermission):
    """
    Custom permission to only allow admins or managers to edit objects.
    """
    def has_permission(self, request, view):
        if request.method in permissions.SAFE_METHODS:
            return request.user.is_authenticated
        return request.user.is_staff or request.user.groups.filter(name='Mar

class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.all().order_by('name')
    serializer_class = ProductSerializer
    permission_classes = [IsAdminOrManager]

# inventory_project/urls.py
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from inventory import views

router = DefaultRouter()
router.register(r'products', views.ProductViewSet)
router.register(r'categories', views.CategoryViewSet) # Assuming CategoryVie

urlpatterns = [
    path('api/', include(router.urls)),
    # ... other urls like admin and auth
]
```

Frontend (React)

The React frontend would be structured similarly to the Hotel System example. It would use `axios` to interact with the Django API endpoints. Key components would include:

- `ProductTable.js`: To display a list of all products with sorting and filtering.
- `ProductModal.js`: A form to add or edit a product.
- `Dashboard.js`: To display key metrics like low-stock items.
- `BarcodeScanner.js`: A component that could use a library like `react-qr-scanner` to interact with the device camera and look up products by barcode via an API call.

2.4 Guidance for Other Frameworks

- Node.js/Express: The structure would be very similar to the Hotel System example. You would define Sequelize models for `Product`, `Category`, etc., and create Express routes and controllers for each resource.
- Laravel: Define Eloquent models for your tables. Use Laravel's resource controllers to quickly scaffold CRUD API endpoints. Laravel's built-in task scheduling could be used for generating daily stock reports.
- ASP.NET Core: Define entity classes and a `DbContext` for Entity Framework Core. Use API Controllers with attributes for routing (`[ApiController]`, `[Route("api/[controller]")]`). You can use `[Authorize(Roles = "Admin,Manager")]` to handle RBAC directly on your controller actions.

3. Hospital Management System

3.1 Functional Requirements

A comprehensive system to manage patient data, doctor schedules, and appointments in a hospital.

- Patient Management: Add, update, delete, and search for patient records (personal details, medical history).
- Doctor Management: Manage doctor details (specialization, availability), assign doctors to patients, and view schedules.
- Appointment Scheduling: Schedule new appointments, update or cancel existing ones, and view upcoming appointments for both patients and doctors.
- Medical Records: Maintain patient medical history and link records to patients and doctors.
- Staff Management: Manage details of other staff like nurses and receptionists, and assign roles.
- Access Control: Role-based access for Admin, Doctor, Receptionist, and Pharmacist.

3.2 Database Design

The database for a hospital management system (HMS) must prioritize data security (due to sensitive patient information) and relational integrity. The design revolves around the `Patient` and `Employee` (for doctors, nurses, etc.) entities. An ER Diagram for an HMS shows the complex interplay between patients, doctors, and appointments.

Entity-Relationship Diagram (ERD) Explanation

- Users: A central table for all system users to handle authentication. It would link to more specific profile tables.
- Patients: Stores all demographic and contact information for patients. This might link to a `User` record if patients can log in.
- Employees: Stores information for all hospital staff (doctors, nurses, receptionists). A `role` field distinguishes them.
- Doctors: A profile table for doctors, extending the `Employees` table with specialization and other professional details.
- Departments: Lists hospital departments (e.g., Cardiology, Pediatrics).
- Appointments: A key transactional table linking a `Patient` to a `Doctor` for a specific date and time.
- MedicalRecords: Stores clinical notes, diagnoses, and treatment plans for each patient encounter. Each record is linked to a patient and often to a specific appointment.
- Prescriptions: Details of medications prescribed during an appointment, linking to `MedicalRecords` and `Patients`.

Database Schema (SQL)

```
-- Users Table for login credentials
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('admin', 'doctor', 'receptionist', 'pharmacist', 'patient') NO
```

```
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Departments Table
CREATE TABLE Departments (
    department_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT
);

-- Employees Table for all staff
CREATE TABLE Employees (
    employee_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT UNIQUE, -- Nullable if not all employees have system access
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    phone_number VARCHAR(20),
    department_id INT,
    hire_date DATE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);

-- Doctors Table (specialization of Employee)
CREATE TABLE Doctors (
    doctor_id INT PRIMARY KEY, -- This is also the employee_id
    specialization VARCHAR(100) NOT NULL,
    license_number VARCHAR(50) UNIQUE,
    FOREIGN KEY (doctor_id) REFERENCES Employees(employee_id)
);

-- Patients Table
CREATE TABLE Patients (
    patient_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT UNIQUE, -- For patients who can log in to a portal
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE NOT NULL,
    gender ENUM('Male', 'Female', 'Other') NOT NULL,
    address TEXT,
    phone_number VARCHAR(20),
    emergency_contact_name VARCHAR(100),
    emergency_contact_phone VARCHAR(20),
```

```
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);

-- Appointments Table
CREATE TABLE Appointments (
    appointment_id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    appointment_date DATETIME NOT NULL,
    purpose VARCHAR(255),
    status ENUM('scheduled', 'completed', 'cancelled', 'no-show') NOT NULL D
    created_by_id INT, -- ID of receptionist or user who booked it
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (created_by_id) REFERENCES Users(user_id)
);

-- MedicalRecords Table
CREATE TABLE MedicalRecords (
    record_id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    appointment_id INT UNIQUE, -- An encounter can be linked to an appointme
    visit_date DATE NOT NULL,
    diagnosis TEXT,
    treatment_plan TEXT,
    notes TEXT,
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (appointment_id) REFERENCES Appointments(appointment_id)
);

-- Indexes
CREATE INDEX idx_appointments_patient_doctor ON Appointments(patient_id, doc
CREATE INDEX idx_appointments_date ON Appointments(appointment_date);
CREATE INDEX idx_medicalrecords_patient ON MedicalRecords(patient_id);
```

3.3 Implementation Guide (ASP.NET Core + React)

For a system handling sensitive data like a Hospital Management System, a strongly-typed and robust backend like ASP.NET Core is an excellent choice. We'll pair it with a

React frontend.

Backend (ASP.NET Core Web API)

1. Project Setup

Use the .NET CLI to create a new Web API project. You can also add authentication scaffolding.

```
dotnet new webapi -n HospitalApi
cd HospitalApi
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

2. Models and DbContext

Define your C# entity classes that map to the database tables. Use data annotations for validation and keys. The `DbContext` class manages the connection and data transactions. Microsoft's tutorial on EF Core is the definitive guide.

```
// Models/Patient.cs
using System.ComponentModel.DataAnnotations;

public class Patient
{
    [Key]
    public int PatientId { get; set; }

    [Required]
    [StringLength(50)]
    public string FirstName { get; set; }

    [Required]
    [StringLength(50)]
    public string LastName { get; set; }

    [Required]
```

```
    public DateTime DateOfBirth { get; set; }

    // Navigation property for appointments
    public ICollection<Appointment> Appointments { get; set; }
}

// Data/HospitalDbContext.cs
using Microsoft.EntityFrameworkCore;

public class HospitalDbContext : DbContext
{
    public HospitalDbContext(DbContextOptions<HospitalDbContext> options) :

    public DbSet<Patient> Patients { get; set; }
    public DbSet<Doctor> Doctors { get; set; }
    public DbSet<Appointment> Appointments { get; set; }
    public DbSet<User> Users { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Configure relationships, e.g., composite keys or table names
        modelBuilder.Entity<User>().HasIndex(u => u.Email).IsUnique();
    }
}
```

3. API Controllers

Create controllers to expose CRUD operations for your entities. Use attribute routing and dependency injection to inject the `DbContext`.

```
// Controllers/PatientsController.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using Microsoft.EntityFrameworkCore;

[ApiController]
[Route("api/[controller]")]
[Authorize(Roles = "Admin,Doctor,Receptionist")] // Protect all actions in t
public class PatientsController : ControllerBase
{
```

```
    private readonly HospitalDbContext _context;

    public PatientsController(HospitalDbContext context)
    {
        _context = context;
    }

    // GET: api/patients
    [HttpGet]
    public async Task<ActionResult<IEnumerable<Patient>>> GetPatients()
    {
        return await _context.Patients.ToListAsync();
    }

    // GET: api/patients/5
    [HttpGet("{id}")]
    public async Task<ActionResult<Patient>> GetPatient(int id)
    {
        var patient = await _context.Patients.FindAsync(id);

        if (patient == null)
        {
            return NotFound();
        }

        return patient;
    }

    // POST: api/patients
    [HttpPost]
    [Authorize(Roles = "Admin,Receptionist")] // More specific authorizatior
    public async Task<ActionResult<Patient>> PostPatient(Patient patient)
    {
        _context.Patients.Add(patient);
        await _context.SaveChangesAsync();

        return CreatedAtAction(nameof(GetPatient), new { id = patient.Patier
    }
}
```

4. Authentication with JWT

Configure JWT Bearer authentication in `Program.cs` (or `Startup.cs` in older .NET versions). This involves setting up ASP.NET Core Identity, configuring it to use JWTs, and defining an endpoint to generate a token upon successful login.

Frontend (React)

The React frontend will be a Single-Page Application (SPA) that consumes the ASP.NET Core API. The structure and approach are similar to the previous examples.

- State Management: For a complex application like an HMS, a global state management library like Redux or Zustand is highly recommended to manage user authentication state, patient data, and appointment schedules across components.
- UI Components:
  - `PatientSearch.js`: A component with a search bar to find patients by name or ID.
  - `AppointmentCalendar.js`: A more complex component, possibly using a library like `react-big-calendar`, to display and manage appointments for a doctor or a patient.
  - `PatientMedicalHistory.js`: A view that lists all past encounters, diagnoses, and prescriptions for a selected patient.
- Security: The React app should securely store the JWT (e.g., in memory or a secure cookie) and handle token expiration and renewal. It should also implement route guards to prevent unauthorized users from accessing pages like the doctor's dashboard.

3.4 Guidance for Other Frameworks

- Node.js/Express: The implementation would follow the pattern described in the Hotel Reservation System, using Sequelize or another ORM to interact with the database. Given the sensitive nature of health data, extra care must be taken with input validation and ensuring all data transfer is over HTTPS.
- Django: Django's built-in admin is a huge advantage for an HMS, as it provides an out-of-the-box interface for admins and receptionists to manage patients, doctors, and appointments with minimal custom development. Django's robust security features (ORM preventing SQLi, CSRF protection) are also a major plus.

- Laravel: Laravel can be used to build a secure and scalable backend. Its powerful middleware system is ideal for implementing fine-grained role-based access control. You could use Laravel's event system to trigger notifications (e.g., appointment reminders).

4. Library Management System

4.1 Functional Requirements

This project aims to automate the operations of a library, from cataloging books to managing member checkouts.

- Book Management: Add new books, update details (title, author), remove books, and search/filter by title or author.
- Member Management: Add new library members, and update or delete their details.
- Circulation: Issue books to members (tracking book ID, member ID, issue date, due date) and record book returns.
- Organization: Organize books by categories.
- Access Control: Role-based access for Admin, Librarian, and Member.

4.2 Database Design

The database design for a library system centers on the `Books` and `Members` entities and the `Transactions` that link them. A normalized design is crucial to avoid data duplication (e.g., storing book titles in every transaction record). This system design article provides a detailed look at the necessary components.

Entity-Relationship Diagram (ERD) Explanation

- Users: A table to manage login credentials for all roles (Admin, Librarian, Member).
- Members: A profile table for library members, linked to the `Users` table. It would store contact info and membership status.
- Books: Contains metadata about each book title (ISBN, title, author, genre).

- BookCopies: Represents individual physical copies of a book. Each copy has a unique barcode and a status (`available`, `borrowed`, `reserved`). This one-to-many relationship from `Books` to `BookCopies` is key to tracking individual items.
- Authors: A separate table for authors to handle cases where a book has multiple authors (many-to-many relationship with `Books`).
- Categories: For classifying books by genre or subject.
- BorrowingTransactions: A ledger that records every checkout and return. It links a `BookCopy` to a `Member` and includes issue date, due date, and return date.

Database Schema (SQL)

```
-- Users Table for authentication
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('admin', 'librarian', 'member') NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Members Table for library user profiles
CREATE TABLE Members (
    member_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone_number VARCHAR(20),
    membership_date DATE NOT NULL,
    status ENUM('active', 'suspended', 'expired') DEFAULT 'active',
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
);

-- Authors Table
CREATE TABLE Authors (
    author_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

-- Categories Table
```

```sql
CREATE TABLE Categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

-- Books Table (for book metadata)
CREATE TABLE Books (
    book_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    isbn VARCHAR(13) UNIQUE,
    publication_year INT,
    category_id INT,
    FOREIGN KEY (category_id) REFERENCES Categories(category_id)
);

-- BookAuthors Junction Table (Many-to-Many)
CREATE TABLE BookAuthors (
    book_id INT NOT NULL,
    author_id INT NOT NULL,
    PRIMARY KEY (book_id, author_id),
    FOREIGN KEY (book_id) REFERENCES Books(book_id) ON DELETE CASCADE,
    FOREIGN KEY (author_id) REFERENCES Authors(author_id) ON DELETE CASCADE
);

-- BookCopies Table (for physical copies)
CREATE TABLE BookCopies (
    copy_id INT AUTO_INCREMENT PRIMARY KEY,
    book_id INT NOT NULL,
    barcode VARCHAR(100) NOT NULL UNIQUE,
    status ENUM('available', 'borrowed', 'reserved', 'lost', 'maintenance')
    FOREIGN KEY (book_id) REFERENCES Books(book_id) ON DELETE CASCADE
);

-- BorrowingTransactions Table
CREATE TABLE BorrowingTransactions (
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    copy_id INT NOT NULL,
    member_id INT NOT NULL,
    issue_date DATE NOT NULL,
    due_date DATE NOT NULL,
    return_date DATE, -- NULL if not yet returned
    fine_amount DECIMAL(5, 2) DEFAULT 0.00,
    FOREIGN KEY (copy_id) REFERENCES BookCopies(copy_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
```

```
);

-- Indexes
CREATE INDEX idx_books_title ON Books(title);
CREATE INDEX idx_bookcopies_status ON BookCopies(status);
CREATE INDEX idx_transactions_member ON BorrowingTransactions(member_id);
```

4.3 Implementation Guide (Laravel + Blade/Vue.js)

Laravel is a powerful PHP framework with an elegant syntax that makes building full-stack applications straightforward. We'll outline a solution using Laravel for the backend, which can serve traditional server-rendered views with Blade or function as an API for a Vue.js frontend.

Backend (Laravel)

1. Project Setup

Use Composer to create a new Laravel project. You can also install a starter kit like Laravel Breeze for authentication scaffolding.

```
composer create-project laravel/laravel library-app
cd library-app
composer require laravel/breeze --dev
php artisan breeze:install # Choose Blade or Vue/React
php artisan migrate
```

Laravel's documentation on authentication is comprehensive and covers starter kits in detail.

2. Models and Migrations

Use Artisan commands to create models and their corresponding migration files.

```
php artisan make:model Book -m
```

```
php artisan make:model Member -m
php artisan make:model BookCopy -m
php artisan make:model BorrowingTransaction -m
```

Define the table schema in the migration files and the relationships (e.g., `hasMany`, `belongsTo`) in the Eloquent model classes.

```
// app/Models/Book.php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Book extends Model
{
    use HasFactory;

    protected $fillable = ['title', 'isbn', 'publication_year'];

    public function authors()
    {
        return $this->belongsToMany(Author::class, 'book_authors');
    }

    public function copies()
    {
        return $this->hasMany(BookCopy::class);
    }
}
```

3. Controllers

Create resource controllers to handle the HTTP requests for your models.

```
php artisan make:controller BookController --resource
```

This command generates a controller with stub methods for `index`, `create`, `store`, `show`, `edit`, `update`, and `destroy`.

```
// app/Http/Controllers/BookController.php
namespace App\Http\Controllers;

use App\Models\Book;
use Illuminate\Http\Request;

class BookController extends Controller
{
    public function __construct()
    {
        // Protect routes based on roles using middleware
        $this->middleware('auth');
        $this->middleware('role:admin,librarian')->except('index', 'show');
    }

    public function index(Request $request)
    {
        $query = Book::query();
        if ($request->has('search')) {
            $query->where('title', 'like', '%' . $request->get('search') . '
        }
        $books = $query->with('authors')->paginate(15);
        return view('books.index', compact('books')); // Using Blade view
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'title' => 'required|string|max:255',
            'isbn' => 'required|string|unique:books,isbn',
            'author_ids' => 'required|array',
        ]);

        $book = Book::create($validated);
        $book->authors()->sync($request->author_ids);

        return redirect()->route('books.index')->with('success', 'Book added
    }
```

```
}
```

4. Routing and Views

Define your web routes in `routes/web.php`. If you're using Blade, you'll create template files in `resources/views`.

```
// routes/web.php
use App\Http\Controllers\BookController;

Route::resource('books', BookController::class);
// ... other routes
```

If you chose Vue.js during the Breeze installation, Laravel is configured to serve a SPA. You would then define API routes in `routes/api.php` and build your frontend components in `resources/js`.

4.4 Guidance for Other Frameworks

- Node.js/Express: The architecture would be similar to the Hotel and Inventory systems. You would define Sequelize models, create Express routes for books, members, and transactions, and implement JWT-based authentication.
- Django: Django is a natural fit. Its ORM easily handles the relationships, and the built-in admin panel provides a ready-made interface for librarians to manage the catalog and members. DRF would be used to expose an API for a React/Vue frontend.
- ASP.NET Core: Use Entity Framework Core to define the models and relationships. Create API controllers for each resource (`Books`, `Members`). ASP.NET Core's powerful middleware pipeline is great for implementing role-based authorization checks. You could use Razor Pages for a server-side rendered application or a separate React/Angular project for the frontend.

5. Restaurant Management System

5.1 Functional Requirements

This system is designed to streamline restaurant operations, from managing the menu to tracking orders and staff.

- Menu Management: Add, update, or delete menu items, including name, category, price, description, and availability.
- Order Management: Place new orders (table number, items, quantity), update or cancel existing orders, and track order status (e.g., "Pending," "Preparing," "Served").
- Table Management: Add and manage tables (number, capacity, availability) and assign them to customers.
- Staff Management: Manage staff details (name, role, shifts, salary) and track their responsibilities.

5.2 Database Design

The database for a restaurant system needs to be fast and reliable, especially for handling real-time orders. The design focuses on `MenuItems`, `Orders`, and `Tables`. Modern approaches might even use GraphQL APIs on top of such a schema for flexible data fetching.

Entity-Relationship Diagram (ERD) Explanation

- Users: Manages accounts for staff (waiter, chef, cashier, admin).
- MenuItems: Stores details for each dish or drink on the menu.
- MenuCategories: Groups menu items (e.g., 'Appetizers', 'Main Courses', 'Desserts').
- Tables: Represents the physical tables in the restaurant, with their status and capacity.
- Orders: A record of each customer order, linked to a table and a staff member (waiter).

- OrderItems: A junction table detailing which menu items and quantities are in each order. This resolves the many-to-many relationship between `Orders` and `MenuItems`.
- Staff: Contains profiles for all employees, linked to the `Users` table for login. Includes roles and shift information.
- Reservations: A separate table to handle future table bookings.

Database Schema (SQL)

```
-- StaffRoles Table
CREATE TABLE StaffRoles (
    role_id INT AUTO_INCREMENT PRIMARY KEY,
    role_name VARCHAR(50) NOT NULL UNIQUE -- e.g., 'Waiter', 'Chef', 'Cashie
);

-- Staff Table
CREATE TABLE Staff (
    staff_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    role_id INT NOT NULL,
    phone_number VARCHAR(20),
    -- For login, this could link to a Users table
    -- user_id INT UNIQUE,
    FOREIGN KEY (role_id) REFERENCES StaffRoles(role_id)
);

-- Tables Table
CREATE TABLE Tables (
    table_id INT AUTO_INCREMENT PRIMARY KEY,
    table_number VARCHAR(10) NOT NULL UNIQUE,
    capacity INT NOT NULL,
    status ENUM('available', 'occupied', 'reserved') NOT NULL DEFAULT 'avail
);

-- MenuCategories Table
CREATE TABLE MenuCategories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT
```

```
);

-- MenuItems Table
CREATE TABLE MenuItems (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    price DECIMAL(10, 2) NOT NULL,
    category_id INT,
    is_available BOOLEAN DEFAULT TRUE,
    image_url VARCHAR(255),
    FOREIGN KEY (category_id) REFERENCES MenuCategories(category_id)
);

-- Orders Table
CREATE TABLE Orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    table_id INT NOT NULL,
    staff_id INT, -- Waiter who took the order
    order_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status ENUM('pending', 'preparing', 'ready', 'served', 'paid', 'cancelle
    total_amount DECIMAL(10, 2),
    FOREIGN KEY (table_id) REFERENCES Tables(table_id),
    FOREIGN KEY (staff_id) REFERENCES Staff(staff_id)
);

-- OrderItems Table (Junction Table)
CREATE TABLE OrderItems (
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    item_id INT NOT NULL,
    quantity INT NOT NULL,
    price_per_item DECIMAL(10, 2) NOT NULL, -- Price at the time of order
    notes TEXT, -- e.g., 'no onions'
    FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON DELETE CASCADE,
    FOREIGN KEY (item_id) REFERENCES MenuItems(item_id)
);

-- Reservations Table
CREATE TABLE Reservations (
    reservation_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    customer_phone VARCHAR(20) NOT NULL,
    table_id INT,
```

```
    reservation_time DATETIME NOT NULL,
    num_guests INT NOT NULL,
    status ENUM('confirmed', 'seated', 'cancelled') DEFAULT 'confirmed',
    FOREIGN KEY (table_id) REFERENCES Tables(table_id)
);

-- Indexes
CREATE INDEX idx_orders_status ON Orders(status);
CREATE INDEX idx_orders_time ON Orders(order_time);
CREATE INDEX idx_menuitems_category ON MenuItems(category_id);
```

5.3 Implementation Guide (Node.js/Express + React)

A real-time system like this benefits from technologies that can handle concurrent connections and push updates to clients. Node.js is excellent for this, and we can use WebSockets (with a library like Socket.IO) to provide live updates to the kitchen and floor staff.

Backend (Node.js, Express, Socket.IO)

1. Project Setup

In addition to the standard Express setup, we'll add `socket.io`.

```
npm install express sequelize mysql2 jsonwebtoken bcryptjs cors dotenv socke
```

2. Real-time Order Tracking with Socket.IO

When an order's status changes (e.g., a waiter places it, a chef marks it as 'preparing'), the backend can emit a WebSocket event. Clients (like a kitchen display system or a waiter's tablet) can listen for these events and update their UI in real-time without needing to poll the server.

```
// server.js
const express = require('express');
```

```
const http = require('http');
const { Server } = require("socket.io");

const app = express();
const server = http.createServer(app);
const io = new Server(server, {
    cors: {
        origin: "http://localhost:3000", // Your React app's URL
        methods: ["GET", "POST"]
    }
});

// Socket.IO connection logic
io.on('connection', (socket) => {
    console.log('A user connected:', socket.id);

    // Join a room, e.g., a kitchen display joins the 'kitchen' room
    socket.on('join_room', (room) => {
        socket.join(room);
        console.log(`User ${socket.id} joined room: ${room}`);
    });

    socket.on('disconnect', () => {
        console.log('User disconnected:', socket.id);
    });
});

// Make `io` available to your controllers
app.set('io', io);

// ... rest of your Express app setup (middleware, routes) ...

server.listen(5000, () => console.log('Server with WebSockets is running on

// In your orderController.js, when an order is created or updated:
const io = req.app.get('io');
// After creating a new order...
io.to('kitchen').emit('new_order', newOrder); // Notify the kitchen
io.to(`waiter_${waiterId}`).emit('order_update', updatedOrder); // Notify th
```

3. API Endpoints

The REST API will handle the standard CRUD operations for menu items, tables, and staff. The order creation endpoint will be the most critical.

```
// controllers/orderController.js
exports.createOrder = async (req, res) => {
    const { table_id, items, staff_id } = req.body; // items is an array of

    // Use a database transaction to ensure atomicity
    const t = await sequelize.transaction();
    try {
        // 1. Create the main order record
        const order = await Order.create({ table_id, staff_id, status: 'penc

        let totalAmount = 0;
        // 2. Create each OrderItem and calculate total
        for (const item of items) {
            const menuItem = await MenuItem.findByPk(item.item_id);
            if (!menuItem) throw new Error(`Menu item with ID ${item.item_ic

            await OrderItem.create({
                order_id: order.order_id,
                item_id: item.item_id,
                quantity: item.quantity,
                price_per_item: menuItem.price,
                notes: item.notes
            }, { transaction: t });
            totalAmount += menuItem.price * item.quantity;
        }

        // 3. Update the order's total amount
        order.total_amount = totalAmount;
        await order.save({ transaction: t });

        // 4. Commit the transaction
        await t.commit();

        // 5. Emit WebSocket event to notify relevant staff
        const io = req.app.get('io');
        const fullOrderDetails = await Order.findByPk(order.order_id, { incl
        io.to('kitchen').emit('new_order', fullOrderDetails); // Send to kit
        io.to('cashier').emit('new_order', fullOrderDetails); // Send to cas
```

```
        res.status(201).json(fullOrderDetails);
    } catch (error) {
        await t.rollback();
        res.status(500).json({ message: 'Failed to create order', error: err
    }
};
```

Frontend (React)

The React frontend would have different views based on user roles.

- Waiter View: A tablet-friendly interface showing a floor plan of tables. Tapping a table allows placing a new order or viewing an existing one. It would listen for `order_update` WebSocket events.
- Kitchen View (KDS - Kitchen Display System): A screen that displays incoming orders as tickets. It would listen for `new_order` events. Chefs could tap an order to change its status to 'Preparing', which would emit another event back to the server to notify the waiter.
- Manager/Admin View: A dashboard for managing the menu, staff, and viewing sales reports.

The frontend would use the `socket.io-client` library to connect to the WebSocket server.

```
// In a component like KitchenDisplay.js
import React, { useEffect, useState } from 'react';
import io from 'socket.io-client';

const socket = io('http://localhost:5000');

const KitchenDisplay = () => {
    const [orders, setOrders] = useState([]);

    useEffect(() => {
        // Join the 'kitchen' room on component mount
        socket.emit('join_room', 'kitchen');

        // Listen for new orders
        socket.on('new_order', (newOrder) => {
```

```
            setOrders((prevOrders) => [newOrder, ...prevOrders]);
        });

        // Clean up on component unmount
        return () => {
            socket.off('new_order');
        };
    }, []);

    return (
        <div>
            <h1>Kitchen Orders</h1>
            {/* Render the list of orders */}
        </div>
    );
};
```



6. HR Management System

6.1 Functional Requirements

A system to centralize and manage employee information and recruitment processes.

- Employee Management: Add, update, and delete employee details (contact info, department, salary). Maintain detailed employee profiles (qualifications, experience).
- Recruitment Management: Track job openings (title, requirements, status) and manage applicant data (resume, application status).
- Data Organization: Store department details and allow filtering of employee records by department or job title.

6.2 Database Design

The database for an HR system must securely handle sensitive Personally Identifiable Information (PII). The schema is centered around the `Employees` entity and its related data, such as performance, attendance, and recruitment.

Entity-Relationship Diagram (ERD) Explanation

- Employees: The central table holding all personal and professional information about an employee.
- Departments: A list of all departments within the organization.
- JobTitles: A list of official job titles (e.g., 'Software Engineer', 'HR Manager').
- Dependents: Stores information about an employee's dependents for benefits administration.
- LeaveRequests: Tracks employee requests for time off.
- PerformanceReviews: Records periodic performance evaluations for each employee.
- JobOpenings: Lists available positions that the company is hiring for.
- Applicants: Stores information about candidates who have applied for jobs.
- Applications: A junction table linking an `Applicant` to a `JobOpening`, tracking the status of their application.

Database Schema (SQL)

```
-- Departments Table
CREATE TABLE Departments (
    department_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT
);

-- JobTitles Table
CREATE TABLE JobTitles (
    job_title_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(100) NOT NULL UNIQUE
);

-- Employees Table
CREATE TABLE Employees (
    employee_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
```

```
    email VARCHAR(100) NOT NULL UNIQUE,
    phone_number VARCHAR(20),
    hire_date DATE NOT NULL,
    termination_date DATE,
    department_id INT,
    job_title_id INT,
    salary DECIMAL(12, 2),
    address TEXT,
    date_of_birth DATE,
    -- For login, could link to a Users table
    -- user_id INT UNIQUE,
    FOREIGN KEY (department_id) REFERENCES Departments(department_id),
    FOREIGN KEY (job_title_id) REFERENCES JobTitles(job_title_id)
);

-- LeaveRequests Table
CREATE TABLE LeaveRequests (
    leave_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT NOT NULL,
    leave_type ENUM('vacation', 'sick', 'personal') NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    status ENUM('pending', 'approved', 'rejected') NOT NULL DEFAULT 'pending
    reason TEXT,
    FOREIGN KEY (employee_id) REFERENCES Employees(employee_id) ON DELETE CA
);

-- JobOpenings Table
CREATE TABLE JobOpenings (
    opening_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    department_id INT,
    description TEXT NOT NULL,
    requirements TEXT,
    status ENUM('open', 'closed', 'on-hold') NOT NULL DEFAULT 'open',
    posted_date DATE,
    FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);

-- Applicants Table
CREATE TABLE Applicants (
    applicant_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
```

```
    email VARCHAR(100) NOT NULL UNIQUE,
    phone_number VARCHAR(20),
    resume_path VARCHAR(255) -- Path to stored resume file
);

-- Applications Table
CREATE TABLE Applications (
    application_id INT AUTO_INCREMENT PRIMARY KEY,
    applicant_id INT NOT NULL,
    opening_id INT NOT NULL,
    application_date DATE NOT NULL,
    status ENUM('applied', 'screening', 'interviewing', 'offered', 'rejected
    notes TEXT,
    FOREIGN KEY (applicant_id) REFERENCES Applicants(applicant_id) ON DELETE
    FOREIGN KEY (opening_id) REFERENCES JobOpenings(opening_id) ON DELETE CA
);

-- Indexes
CREATE INDEX idx_employees_department ON Employees(department_id);
CREATE INDEX idx_employees_job_title ON Employees(job_title_id);
CREATE INDEX idx_applications_status ON Applications(status);
```



7. Online Rental System

7.1 Functional Requirements

A platform for users to list and rent items, similar to a peer-to-peer rental marketplace.

- User Actions: Browse rental listings, search/filter by category/price/location, and view item details (images, description, price).
- Booking: Book rental items for selected dates, view the total cost, and view/cancel existing bookings.
- Owner Actions: Add, edit, or delete rental items. Manage booking requests (approve/decline).
- Admin Actions: Register an account and manage all users (suspend/activate).

7.2 Database Design

The database needs to manage users, the items they own, and the bookings made by other users. The core of the system is the `Items` table and the `Bookings` table, which tracks availability.

Entity-Relationship Diagram (ERD) Explanation

- Users: Stores all users, both item owners and renters. A role system (`user`, `admin`) is needed.
- Items: Contains all information about a rentable item, including a link to its `owner` (a User).
- ItemCategories: For classifying items (e.g., 'Electronics', 'Tools', 'Vehicles').
- ItemImages: A separate table to allow multiple images per item.
- Bookings: Records a rental transaction, linking an `item` to a `renter` (a User) for a specific `start_date` and `end_date`.
- Reviews: Allows renters to leave reviews for items they have booked.

Database Schema (SQL)

```
-- Users Table
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(100),
    role ENUM('user', 'admin') NOT NULL DEFAULT 'user',
    status ENUM('active', 'suspended') NOT NULL DEFAULT 'active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- ItemCategories Table
CREATE TABLE ItemCategories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
```

```
    description TEXT
);

-- Items Table
CREATE TABLE Items (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    owner_id INT NOT NULL,
    category_id INT,
    title VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    price_per_day DECIMAL(10, 2) NOT NULL,
    location VARCHAR(255),
    is_available BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (owner_id) REFERENCES Users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (category_id) REFERENCES ItemCategories(category_id)
);

-- ItemImages Table
CREATE TABLE ItemImages (
    image_id INT AUTO_INCREMENT PRIMARY KEY,
    item_id INT NOT NULL,
    image_url VARCHAR(255) NOT NULL,
    is_primary BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (item_id) REFERENCES Items(item_id) ON DELETE CASCADE
);

-- Bookings Table
CREATE TABLE Bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    item_id INT NOT NULL,
    renter_id INT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    total_cost DECIMAL(10, 2) NOT NULL,
    status ENUM('pending', 'approved', 'declined', 'completed', 'cancelled')
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (item_id) REFERENCES Items(item_id),
    FOREIGN KEY (renter_id) REFERENCES Users(user_id),
    CONSTRAINT chk_booking_dates CHECK (end_date >= start_date)
);

-- Reviews Table
CREATE TABLE Reviews (
```

```
    review_id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT NOT NULL UNIQUE, -- One review per booking
    reviewer_id INT NOT NULL,
    item_id INT NOT NULL,
    rating INT NOT NULL CHECK (rating >= 1 AND rating <= 5),
    comment TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (booking_id) REFERENCES Bookings(booking_id),
    FOREIGN KEY (reviewer_id) REFERENCES Users(user_id),
    FOREIGN KEY (item_id) REFERENCES Items(item_id)
);

-- Indexes
CREATE INDEX idx_items_category ON Items(category_id);
CREATE INDEX idx_items_location ON Items(location);
CREATE INDEX idx_bookings_item_dates ON Bookings(item_id, start_date, end_da
```

> Concurrency Challenge: A critical challenge in this system is preventing double bookings. When a user requests to book an item, you must check for overlapping date ranges for that item. This check and the subsequent creation of the booking must be done in a single, atomic database transaction to prevent race conditions where two users book the same item for the same dates simultaneously.



8. Tourist Management System

8.1 Functional Requirements

A platform connecting tourists with service providers (e.g., tour guides, hotels).

- Roles: Create roles for Tourist (Guest/Registered), Service Provider (Business), and Administrator.
- Discovery: Tourists can search/filter listings by location, category, and rating, and view detailed listings (photos, prices, hours).

- Service Provider Actions: Add/update services (tours, hotels, activities), and manage booking requests (confirm/decline).
- Admin Actions: View a dashboard of all bookings and their statuses.

8.2 Database Design

This system is a marketplace, so the database must model the relationship between providers, the services they offer, and the tourists who book them.

Entity-Relationship Diagram (ERD) Explanation

- Users: A central table for all user types, differentiated by a `role` field ('tourist', 'provider', 'admin').
- ServiceProviders: A profile table for businesses, linked to a `User` account. Contains business name, description, etc.
- Services: The core entity representing what providers offer (e.g., 'City Walking Tour', 'Deluxe Hotel Room'). Each service is linked to a `ServiceProvider`.
- ServiceCategories: To classify services (e.g., 'Tours', 'Accommodation', 'Activities').
- Bookings: Records a transaction where a `Tourist` (User) books a `Service`.
- Reviews: Allows tourists to rate and review services they have used.

Database Schema (SQL)

```
-- Users Table
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(100),
    role ENUM('tourist', 'provider', 'admin') NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- ServiceProviders Table (Business Profiles)
CREATE TABLE ServiceProviders (
```

```
    provider_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE, -- Link to the user account
    business_name VARCHAR(150) NOT NULL,
    description TEXT,
    address TEXT,
    phone_number VARCHAR(20),
    is_verified BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
);

-- ServiceCategories Table
CREATE TABLE ServiceCategories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT
);

-- Services Table
CREATE TABLE Services (
    service_id INT AUTO_INCREMENT PRIMARY KEY,
    provider_id INT NOT NULL,
    category_id INT NOT NULL,
    title VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    location VARCHAR(255),
    price DECIMAL(10, 2), -- Can be per person, per night, etc.
    price_unit VARCHAR(50), -- e.g., 'per_person', 'per_night'
    average_rating DECIMAL(3, 2) DEFAULT 0.00,
    FOREIGN KEY (provider_id) REFERENCES ServiceProviders(provider_id) ON DE
    FOREIGN KEY (category_id) REFERENCES ServiceCategories(category_id)
);

-- ServiceImages Table
CREATE TABLE ServiceImages (
    image_id INT AUTO_INCREMENT PRIMARY KEY,
    service_id INT NOT NULL,
    image_url VARCHAR(255) NOT NULL,
    caption VARCHAR(255),
    FOREIGN KEY (service_id) REFERENCES Services(service_id) ON DELETE CASCA
);

-- Bookings Table
CREATE TABLE Bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    service_id INT NOT NULL,
    tourist_id INT NOT NULL,
    booking_date DATE NOT NULL,
    num_participants INT DEFAULT 1,
    total_price DECIMAL(10, 2) NOT NULL,
    status ENUM('pending', 'confirmed', 'declined', 'completed', 'cancelled'
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (service_id) REFERENCES Services(service_id),
    FOREIGN KEY (tourist_id) REFERENCES Users(user_id)
);

-- Reviews Table
CREATE TABLE Reviews (
    review_id INT AUTO_INCREMENT PRIMARY KEY,
    service_id INT NOT NULL,
    tourist_id INT NOT NULL,
    booking_id INT, -- Optional: link review to a specific booking
    rating INT NOT NULL CHECK (rating >= 1 AND rating <= 5),
    comment TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (service_id) REFERENCES Services(service_id),
    FOREIGN KEY (tourist_id) REFERENCES Users(user_id),
    FOREIGN KEY (booking_id) REFERENCES Bookings(booking_id)
);

-- Indexes
CREATE INDEX idx_services_location ON Services(location);
CREATE INDEX idx_services_category ON Services(category_id);
CREATE INDEX idx_bookings_status ON Bookings(status);
```



9. School Management System

9.1 Functional Requirements

A comprehensive system for managing academic and administrative activities in a school.

- Academic Structure: Manage academic years, create classes/batches, and handle subject allocation.

- User Management: Create, update, and deactivate profiles for students, teachers, and staff. Assign system roles and access levels.
- Student-Specific Features: Students can view their personal timetable, assigned subjects, and teachers.
- Admin-Specific Features: Admins can view system-wide reports, dashboards, and audit logs.

9.2 Database Design

A School Management System (SMS) requires a highly structured and relational database to manage the complex relationships between students, teachers, classes, subjects, and schedules. Data integrity is paramount. This guide on building a school management schema provides a detailed overview of the necessary entities.

Entity-Relationship Diagram (ERD) Explanation

- Users: A base table for all individuals with system access (students, teachers, staff, admins).
- Students: Profile information for students, linked to a `User` account.
- Teachers: Profile information for teachers, linked to a `User` account.
- AcademicYears: Defines academic sessions (e.g., '2025-2026').
- Classes: Represents a grade or batch (e.g., 'Grade 10 - Section A'), linked to an `AcademicYear`.
- Subjects: A list of all subjects taught in the school (e.g., 'Mathematics', 'Physics').
- ClassSubjects: A junction table that assigns `Subjects` to `Classes` and assigns a `Teacher` to teach that subject for that class.
- Enrollments: A junction table that enrolls `Students` into a `Class` for a specific `AcademicYear`.
- Timetable: Stores the weekly schedule, linking a `ClassSubject` to a specific day and time slot.
- AuditLogs: A critical table for admins to track important actions performed in the system (e.g., who changed a student's grade).

Database Schema (SQL)

```
-- Users Table for authentication
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('admin', 'teacher', 'student', 'staff') NOT NULL,
    is_active BOOLEAN DEFAULT TRUE
);

-- Students Table
CREATE TABLE Students (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE,
    admission_number VARCHAR(20) UNIQUE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
);

-- Teachers Table
CREATE TABLE Teachers (
    teacher_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    qualification VARCHAR(255),
    hire_date DATE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
);

-- AcademicYears Table
CREATE TABLE AcademicYears (
    year_id INT AUTO_INCREMENT PRIMARY KEY,
    year_name VARCHAR(20) NOT NULL UNIQUE, -- e.g., '2025-2026'
    start_date DATE NOT NULL,
    end_date DATE NOT NULL
);

-- Classes Table (e.g., Grade 10, Section A)
```

```
CREATE TABLE Classes (
    class_id INT AUTO_INCREMENT PRIMARY KEY,
    class_name VARCHAR(50) NOT NULL, -- e.g., 'Grade 10 - A'
    academic_year_id INT NOT NULL,
    class_teacher_id INT, -- Homeroom teacher
    FOREIGN KEY (academic_year_id) REFERENCES AcademicYears(year_id),
    FOREIGN KEY (class_teacher_id) REFERENCES Teachers(teacher_id)
);

-- Subjects Table
CREATE TABLE Subjects (
    subject_id INT AUTO_INCREMENT PRIMARY KEY,
    subject_name VARCHAR(100) NOT NULL UNIQUE,
    subject_code VARCHAR(20) UNIQUE
);

-- Enrollments Table (Students in Classes)
CREATE TABLE Enrollments (
    enrollment_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    class_id INT NOT NULL,
    enrollment_date DATE NOT NULL,
    UNIQUE(student_id, class_id),
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (class_id) REFERENCES Classes(class_id)
);

-- ClassSubjects Table (Assigning subjects and teachers to classes)
CREATE TABLE ClassSubjects (
    class_subject_id INT AUTO_INCREMENT PRIMARY KEY,
    class_id INT NOT NULL,
    subject_id INT NOT NULL,
    teacher_id INT NOT NULL,
    UNIQUE(class_id, subject_id),
    FOREIGN KEY (class_id) REFERENCES Classes(class_id),
    FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id),
    FOREIGN KEY (teacher_id) REFERENCES Teachers(teacher_id)
);

-- Timetable Table
CREATE TABLE Timetable (
    timetable_id INT AUTO_INCREMENT PRIMARY KEY,
    class_subject_id INT NOT NULL,
    day_of_week ENUM('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday')
```

```
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    room_number VARCHAR(20),
    FOREIGN KEY (class_subject_id) REFERENCES ClassSubjects(class_subject_id
);

-- AuditLogs Table
CREATE TABLE AuditLogs (
    log_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    action VARCHAR(255) NOT NULL, -- e.g., 'UPDATED_STUDENT_PROFILE'
    target_entity VARCHAR(50), -- e.g., 'Student'
    target_id INT,
    details TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```



10. Online Food Ordering System

10.1 Functional Requirements

A multi-sided platform connecting customers, restaurants, and delivery drivers.

- Customer Actions: Browse restaurants and menus, search/filter by price/ratings, place orders for delivery/pickup, view order history, and rate/review orders.
- Restaurant Actions: Manage restaurant profile and menu, receive and manage orders, and update order status (accepted, preparing, etc.).
- Driver Actions: View assigned delivery orders and accept/reject delivery requests.
- Admin Actions: Manage all users (customers, restaurants, drivers) and generate reports.

10.2 Database Design

This is a complex system with three main user types and real-time dependencies. The database must handle relationships between restaurants, their menus, customer orders,

and the delivery logistics.

Entity-Relationship Diagram (ERD) Explanation

- Users: A unified table for all actors (customer, restaurant_owner, driver, admin) with a `role` to distinguish them.
- Restaurants: Profile information for each restaurant, linked to a `User` (the owner).
- MenuItems: The menu for each restaurant.
- Orders: The central entity tracking a customer's order from a restaurant. It must have a clear status lifecycle.
- OrderItems: A junction table for items within an order.
- Deliveries: A table to manage the logistics of an order, linking an `Order` to a `Driver` and tracking delivery status.
- Reviews: For customers to rate restaurants and/or specific orders.

Database Schema (SQL)

```
-- Users Table
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(100),
    phone_number VARCHAR(20) NOT NULL,
    role ENUM('customer', 'restaurant_owner', 'driver', 'admin') NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Restaurants Table
CREATE TABLE Restaurants (
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,
    owner_id INT NOT NULL UNIQUE,
    name VARCHAR(150) NOT NULL,
    address TEXT NOT NULL,
    cuisine_type VARCHAR(100),
    average_rating DECIMAL(3, 2) DEFAULT 0.00,
```

```
    is_open BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (owner_id) REFERENCES Users(user_id) ON DELETE CASCADE
);

-- MenuItems Table
CREATE TABLE MenuItems (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_id INT NOT NULL,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    price DECIMAL(10, 2) NOT NULL,
    is_available BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (restaurant_id) REFERENCES Restaurants(restaurant_id) ON DEL
);

-- Orders Table
CREATE TABLE Orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    restaurant_id INT NOT NULL,
    order_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(10, 2) NOT NULL,
    status ENUM('pending', 'accepted', 'preparing', 'ready_for_pickup', 'out
    order_type ENUM('delivery', 'pickup') NOT NULL,
    delivery_address TEXT,
    FOREIGN KEY (customer_id) REFERENCES Users(user_id),
    FOREIGN KEY (restaurant_id) REFERENCES Restaurants(restaurant_id)
);

-- OrderItems Table
CREATE TABLE OrderItems (
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    item_id INT NOT NULL,
    quantity INT NOT NULL,
    price_per_item DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON DELETE CASCADE,
    FOREIGN KEY (item_id) REFERENCES MenuItems(item_id)
);

-- Deliveries Table
CREATE TABLE Deliveries (
    delivery_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL UNIQUE,
```

```
    driver_id INT, -- Can be NULL until a driver accepts
    status ENUM('pending_assignment', 'accepted', 'picked_up', 'delivered',
    pickup_time TIMESTAMP,
    delivery_time TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (driver_id) REFERENCES Users(user_id)
);

-- Reviews Table
CREATE TABLE Reviews (
    review_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    customer_id INT NOT NULL,
    restaurant_id INT NOT NULL,
    rating INT NOT NULL CHECK (rating >= 1 AND rating <= 5),
    comment TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (customer_id) REFERENCES Users(user_id),
    FOREIGN KEY (restaurant_id) REFERENCES Restaurants(restaurant_id)
);

-- Indexes
CREATE INDEX idx_orders_customer ON Orders(customer_id);
CREATE INDEX idx_orders_restaurant ON Orders(restaurant_id);
CREATE INDEX idx_deliveries_driver ON Deliveries(driver_id);
```



11. Blood Bank Management System

11.1 Functional Requirements

A critical system for managing blood donations, inventory, and requests.

- Donor Management: Donor registration and profile management. Donors can view their donation history and blood test results.
- Inventory Management: Search for specific blood types and view availability. Generate blood compatibility reports.
- Request Management: Hospitals or patients can register blood requests.

- Admin Management: User management for donors, staff, and hospitals. Generate blood availability reports.

11.2 Database Design

The database for a blood bank must ensure accuracy and traceability. The core entities are `Donors`, `BloodBags` (the inventory), and `Requests`.

Entity-Relationship Diagram (ERD) Explanation

- Users: Manages login accounts for donors, staff, and hospital representatives.
- Donors: Detailed profiles for individuals who donate blood, linked to a `User` account. Includes blood type and medical history.
- Hospitals: Profiles for registered hospitals that can request blood.
- Donations: A record of each donation event, linking a `Donor` to a specific date and location.
- BloodBags: The inventory. Each bag is created from a `Donation`, has a unique ID, blood type, collection date, and expiry date.
- BloodRequests: A record of a request for blood made by a `Hospital` or patient.
- Dispatch: A table that links a `BloodBag` to a `BloodRequest` when inventory is issued.

Database Schema (SQL)

```
-- Users Table
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('donor', 'staff', 'hospital', 'admin') NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Donors Table
CREATE TABLE Donors (
```

```
    donor_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    full_name VARCHAR(100) NOT NULL,
    date_of_birth DATE NOT NULL,
    blood_type ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-') NOT NU
    phone_number VARCHAR(20),
    address TEXT,
    last_donation_date DATE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
);

-- Hospitals Table
CREATE TABLE Hospitals (
    hospital_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE, -- User account for the hospital representa
    name VARCHAR(150) NOT NULL,
    address TEXT NOT NULL,
    phone_number VARCHAR(20),
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
);

-- Donations Table
CREATE TABLE Donations (
    donation_id INT AUTO_INCREMENT PRIMARY KEY,
    donor_id INT NOT NULL,
    donation_date DATE NOT NULL,
    location VARCHAR(255), -- e.g., 'Main Blood Center', 'Mobile Drive X'
    hemoglobin_level DECIMAL(4, 1),
    blood_pressure VARCHAR(20),
    notes TEXT,
    FOREIGN KEY (donor_id) REFERENCES Donors(donor_id)
);

-- BloodBags Table (Inventory)
CREATE TABLE BloodBags (
    bag_id INT AUTO_INCREMENT PRIMARY KEY,
    donation_id INT NOT NULL UNIQUE,
    blood_type ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-') NOT NU
    collection_date DATE NOT NULL,
    expiry_date DATE NOT NULL,
    status ENUM('available', 'reserved', 'dispatched', 'expired') NOT NULL D
    storage_location VARCHAR(100),
    FOREIGN KEY (donation_id) REFERENCES Donations(donation_id)
);
```

```
-- BloodRequests Table
CREATE TABLE BloodRequests (
    request_id INT AUTO_INCREMENT PRIMARY KEY,
    requester_id INT NOT NULL, -- Can be a hospital user or a patient user
    patient_name VARCHAR(100),
    required_blood_type ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-
    quantity_ml INT NOT NULL,
    urgency ENUM('low', 'medium', 'high', 'critical') NOT NULL,
    status ENUM('pending', 'fulfilled', 'partially_fulfilled', 'cancelled')
    request_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (requester_id) REFERENCES Users(user_id)
);

-- Dispatch Table (Fulfilling requests)
CREATE TABLE Dispatch (
    dispatch_id INT AUTO_INCREMENT PRIMARY KEY,
    bag_id INT NOT NULL UNIQUE,
    request_id INT NOT NULL,
    dispatch_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    dispatched_by_staff_id INT NOT NULL,
    FOREIGN KEY (bag_id) REFERENCES BloodBags(bag_id),
    FOREIGN KEY (request_id) REFERENCES BloodRequests(request_id),
    FOREIGN KEY (dispatched_by_staff_id) REFERENCES Users(user_id)
);

-- Indexes
CREATE INDEX idx_bloodbags_type_status ON BloodBags(blood_type, status);
CREATE INDEX idx_bloodbags_expiry ON BloodBags(expiry_date);
CREATE INDEX idx_donations_donor ON Donations(donor_id);
```



12. Pharmacy Management System

12.1 Functional Requirements

A system for managing medicine inventory, sales, and customer data in a pharmacy.

- Medicine Management: Add, update, and delete medicine records. Search for medicines by name, type, or condition.

- Inventory Control: Stock alerts for low inventory and tracking of expiry dates. Barcode/QR code scanning.
- Sales & Reporting: Process sales, generate sales reports, and calculate profit/loss.
- User Management: Manage profiles for customers, staff, and suppliers.

12.2 Database Design

The database must be efficient at tracking stock levels, sales transactions, and expiry dates. It shares similarities with the general Inventory Management System but is specialized for pharmaceutical products.

Entity-Relationship Diagram (ERD) Explanation

- Users: Accounts for customers, staff (pharmacist, cashier), and admins.
- Medicines: The main catalog of all pharmaceutical products sold. Includes details like manufacturer, composition, etc.
- Suppliers: Information about pharmaceutical distributors.
- Stock (or Batches): This is a crucial table. Instead of just a `quantity` on the `Medicines` table, this table tracks individual batches of a medicine, each with its own `batch_number`, `expiry_date`, `quantity`, and `cost_price`. This is essential for expiry tracking and profit calculation.
- Sales: Records a single sales transaction.
- SaleItems: A junction table detailing which medicine `batches` were sold in which `sale`.
- PurchaseOrders: To manage orders from suppliers.

Database Schema (SQL)

```
-- Users Table
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
```

```
    role ENUM('admin', 'pharmacist', 'cashier', 'customer') NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Suppliers Table
CREATE TABLE Suppliers (
    supplier_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    contact_person VARCHAR(100),
    phone_number VARCHAR(20),
    email VARCHAR(100)
);

-- Medicines Table (Catalog)
CREATE TABLE Medicines (
    medicine_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    manufacturer VARCHAR(100),
    category VARCHAR(100), -- e.g., 'Antibiotic', 'Painkiller'
    description TEXT,
    requires_prescription BOOLEAN DEFAULT FALSE
);

-- StockBatches Table (Inventory)
CREATE TABLE StockBatches (
    batch_id INT AUTO_INCREMENT PRIMARY KEY,
    medicine_id INT NOT NULL,
    batch_number VARCHAR(50) UNIQUE,
    expiry_date DATE NOT NULL,
    quantity INT NOT NULL,
    cost_price_per_unit DECIMAL(10, 2) NOT NULL,
    selling_price_per_unit DECIMAL(10, 2) NOT NULL,
    supplier_id INT,
    received_date DATE,
    barcode VARCHAR(100),
    FOREIGN KEY (medicine_id) REFERENCES Medicines(medicine_id),
    FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id)
);

-- Sales Table
CREATE TABLE Sales (
    sale_id INT AUTO_INCREMENT PRIMARY KEY,
    staff_id INT NOT NULL, -- User who made the sale
    customer_id INT, -- Optional, for registered customers
```

```
    sale_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(10, 2) NOT NULL,
    payment_method VARCHAR(50),
    FOREIGN KEY (staff_id) REFERENCES Users(user_id),
    FOREIGN KEY (customer_id) REFERENCES Users(user_id)
);

-- SaleItems Table
CREATE TABLE SaleItems (
    sale_item_id INT AUTO_INCREMENT PRIMARY KEY,
    sale_id INT NOT NULL,
    batch_id INT NOT NULL, -- Link to the specific batch sold
    quantity INT NOT NULL,
    price_per_unit DECIMAL(10, 2) NOT NULL, -- Price at time of sale
    FOREIGN KEY (sale_id) REFERENCES Sales(sale_id) ON DELETE CASCADE,
    FOREIGN KEY (batch_id) REFERENCES StockBatches(batch_id)
);

-- Alerts Table (for low stock and expiry)
CREATE TABLE Alerts (
    alert_id INT AUTO_INCREMENT PRIMARY KEY,
    alert_type ENUM('low_stock', 'expiry_warning') NOT NULL,
    medicine_id INT,
    batch_id INT,
    message TEXT NOT NULL,
    is_dismissed BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (medicine_id) REFERENCES Medicines(medicine_id),
    FOREIGN KEY (batch_id) REFERENCES StockBatches(batch_id)
);

-- Indexes
CREATE INDEX idx_stockbatches_expiry ON StockBatches(expiry_date);
CREATE INDEX idx_stockbatches_medicine ON StockBatches(medicine_id);
CREATE INDEX idx_medicines_name ON Medicines(name);
```

> System Logic: A background job (e.g., a cron job or scheduled task) should run daily to check for medicines approaching their expiry date and for stock levels

> falling below a reorder threshold. When found, it should insert records into the `Alerts` table, which can be displayed on the pharmacist's dashboard.

Reference

[1] sql server - What columns generally make good indexes?
https://stackoverflow.com/questions/107132/what-columns-generally-make-good-indexes

[2] How to Build a Full-Stack React CRUD App
https://medium.com/@corbado_tech/how-to-build-a-full-stack-react-crud-app-with-node-js-express-and-postgresql-f97960b0e467

[3] React + Node.js Express: User Authentication with JWT ...
https://www.bezkoder.com/react-express-authentication-jwt/

[4] rkrishdev/react-node-express-mysql-jwt-boilerplate
https://github.com/rkrishdev/react-node-express-mysql-jwt-boilerplate

[5] DBMS_Project_for_CSE_3rd_Year (1)
https://api.skywork.ai/chat/chat/upload_file?file_id=1999157529783296000

[6] ravikumarxworkz/hospital-management-system
https://github.com/ravikumarxworkz/hospital-management-system

[7] Get started with EF Core in an ASP.NET MVC web app
https://learn.microsoft.com/en-us/aspnet/core/data/ef-mvc/intro?view=aspnetcore-10.0

[8] Implement role-based access control in applications
https://learn.microsoft.com/en-us/entra/identity-platform/howto-implement-rbac-for-apps

[9] Authentication - Laravel 12.x - The PHP Framework For ...
https://laravel.com/docs/12.x/authentication

[10] MERN stack CRUD app in Node.js, Express, React & ...
https://www.corbado.com/blog/react-express-crud-app-mongodb

[11] ravikumarxworkz/hospital-management-system
https://github.com/ravikumarxworkz/hospital-management-system

[12] Best practices to protect your Flask applications
https://escape.tech/blog/best-practices-protect-flask-applications/

[13] How to Design a Database for Online Hotel Booking App ...
https://www.geeksforgeeks.org/dbms/how-to-design-a-database-for-online-hotel-booking-app-like-makemytrip/

[14] ER Diagram for a Hospital Management System
https://www.geeksforgeeks.org/sql/how-to-design-er-diagram-for-a-hospital-management-system/

[15] How to Design Database Inventory Management Systems

https://www.geeksforgeeks.org/dbms/how-to-design-database-inventory-management-systems/

[16] 10 ER Diagrams for a University Management System + ...
https://creately.com/guides/er-diagrams-for-a-university-management-system/

[17] Building a Restaurant Management System with Neurelo ...
https://docs.neurelo.com/tutorials/building-a-restaurant-management-system-with-neurelo-and-mongodb-using-graphql-in-just-a-few-minutes

[18] Designing a Library Management System | by bugfree.ai
https://medium.com/@bugfreeai/system-design-designing-a-library-management-system-56b04a4edcf1

[19] Best Practices to Implement Role-Based Access Control ...
https://www.permit.io/blog/best-practices-to-implement-rbac-for-developers

[20] Authentication - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[21] Boost Query Performance with Database Indexing
https://www.acceldata.io/blog/mastering-database-indexing-strategies-for-peak-performance

[22] File Upload - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[23] Database Normalization: 1NF, 2NF, 3NF & BCNF Examples
https://www.digitalocean.com/community/tutorials/database-normalization

[24] How to Build a Database Schema for School Management ...
https://www.back4app.com/tutorials/how-to-build-a-database-schema-for-school-management-software

[25] Fixing SQL Injection Vulnerabilities in Flask (Python)
https://www.stackhawk.com/blog/finding-and-fixing-sql-injection-vulnerabilities-in-flask-python/

[26] OWASP Top Ten Web Application Security Risks
https://owasp.org/www-project-top-ten/

[27] Server-side Sessions in Flask with Redis
https://testdriven.io/blog/flask-server-side-sessions/

[28] Chapter 8 The Entity Relationship Data Model
https://opentextbc.ca/dbdesign01/chapter/chapter-8-entity-relationship-model/

[29] Building a CRUD API with Laravel and Sanctum
https://dev.to/neon-postgres/building-a-crud-api-with-laravel-and-sanctum-3525

[30] Develop a CRUD Application using Django and React ...

https://medium.com/@fizaashraf37/develop-a-crud-application-using-django-and-react-part-1-2e8927a1e0c8

[31] Cross-site scripting (XSS) - Security - MDN Web Docs
https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/XSS

[32] Basic Swagger Setup for Spring Boot Application
https://medium.com/@samuelgbenga972/basic-swagger-setup-for-spring-boot-application-755adc6e5a9b

[33] Session Management - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[34] Build a Secure CRUD App with ASP.NET Core and React
https://developer.okta.com/blog/2018/07/02/build-a-secure-crud-app-with-aspnetcore-and-react

[35] Database design basics
https://support.microsoft.com/en-us/office/database-design-basics-eb2159cf-1e30-401a-8084-bd4f9c9ca1f5